JuriPredict AI — Guide de développement Frontend

Oct 28, 2026 · @Altes Phanor

1. Projet et rôle du dev frontend

Le dev frontend livre 4 écrans prototypes (Accueil, Assistant IA, Résultat, Détail d'article) branchés sur l'API FastAPI en React JS : construits sur données fictives dès l'heure 4, puis reliés à l'API réelle à partir de l'heure 30 du hackathon de 48 h.

JuriPredict AI analyse une situation décrite en langage naturel, retrouve les articles pertinents du droit haïtien (Constitution, Code pénal, Code civil) via RAG et affiche une analyse structurée avec sources vérifiables.

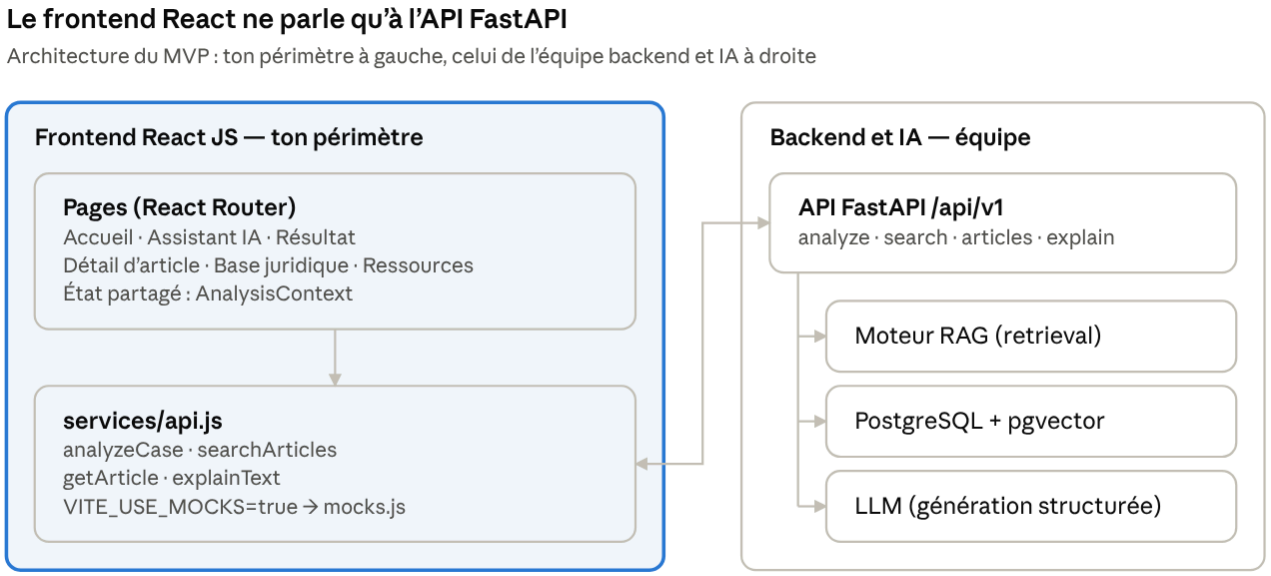
Responsabilités (Membre 3 — Frontend Engineer, cahier des charges §36)

- React JS (Vite), UI, expérience utilisateur, dashboard.
- Séparer visuellement : faits fournis, analyse IA, texte juridique officiel.
- Afficher l'avertissement juridique (F10) sur toute analyse.
- Rendre chaque affirmation juridique reliée à une source consultable (F07).

Parcours utilisateur à respecter (§28) : Décrire → Analyser → Comprendre → Vérifier → Explorer. Jamais de long bloc de texte : des cartes de résultat.

Critère de réussite (§38) : l'utilisateur décrit un cas, obtient une analyse, voit les articles, lit une explication et vérifie les sources.

2. Vue d'ensemble de l'architecture



Architecture JuriPredict AI · frontend React et backend

Les pages appellent uniquement `services/api.js`, qui envoie des requêtes HTTP/JSON à FastAPI ; tant que le backend n'est pas prêt, ce même fichier renvoie les mocks.

3. Stack React JS et structure du dépôt

Le frontend est une application React JS (JavaScript + JSX) créée avec Vite ; elle ne parle qu'à l'API FastAPI. Le cahier des charges recommande Next.js (§21) : l'équipe choisit React JS + Vite, plus simple à prendre en main pendant 48 h.

Couche	Outil	Usage dans JuriPredict
Bibliothèque UI	React 18 (JSX)	Composants et état des écrans
Outil de build	Vite	Serveur de dev rapide, build de production
Navigation	React Router	Routes `/`, `/assistant`, `/resultat`, `/base-juridique/:id`
Style	Tailwind CSS	Palette bleu nuit / bleu / turquoise / alerte
Icônes	lucide-react	Balance, Search, BookOpen, AlertTriangle
Appels API	`fetch` natif	`/api/v1/analyze`, `/search`, `/articles`, `/explain`

Couche	Outil	Usage dans JuriPredict
Partage d'état	Context API (`AnalysisContext`)	Garder l'analyse entre Assistant et Résultat

Structure du dépôt `juripredict-frontend/`

```
juripredict-frontend/
├── index.html
├── .env                          # VITE_API_URL, VITE_USE_MOCKS
├── vite.config.js                # Plugins React + Tailwind
└── src/
    ├── main.jsx                  # Point d'entrée + BrowserRouter
    ├── App.jsx                   # Déclaration des routes
    ├── index.css                 # Directives Tailwind
    ├── pages/
    │   ├── HomePage.jsx          # Écran 1 — Accueil
    │   ├── AssistantPage.jsx     # Écran 2 — Assistant IA
    │   ├── ResultPage.jsx        # Écran 3 — Résultat d'analyse
    │   ├── ArticlePage.jsx       # Écran 4 — Détail d'un article
    │   ├── SearchPage.jsx        # Base juridique
    │   └── ResourcesPage.jsx     # Guide, FAQ, limites
    ├── components/
    │   ├── layout/               # Header, Footer, LegalDisclaimer
    │   ├── analysis/             # FactsCard, QualificationCard,
ScenarioTabs, LimitsBox
    │   └── search/               # SearchBar, ArticleItem, ExplainButton
    ├── context/
    │   └── AnalysisContext.jsx
    └── services/
        ├── api.js                # Client API
        └── mocks.js              # Réponses fictives
```

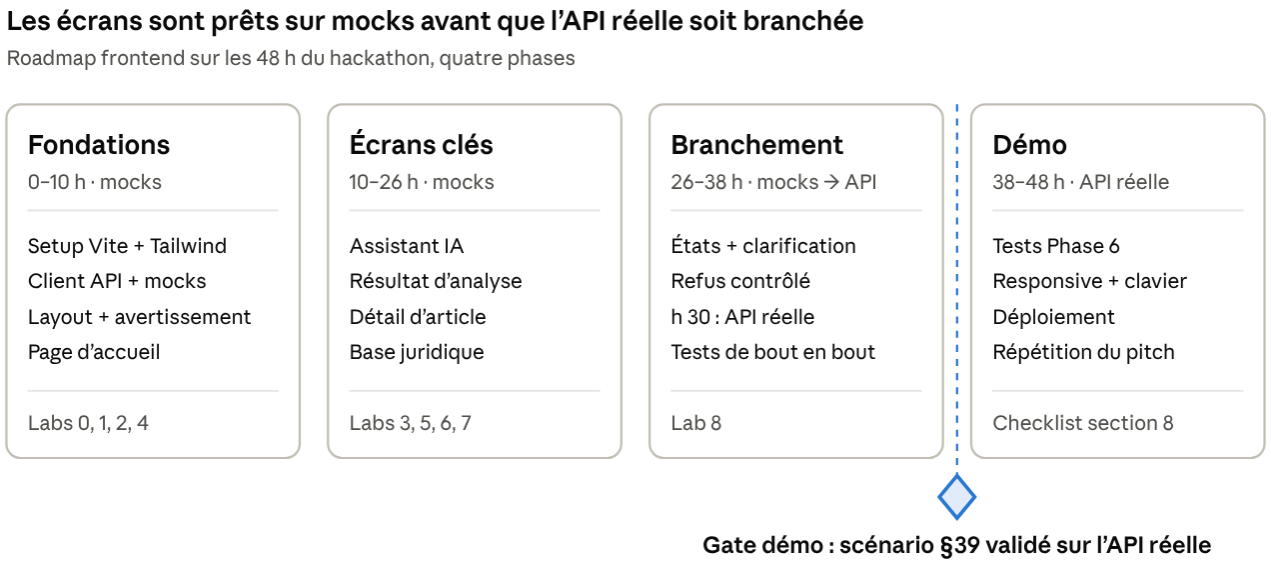
Palette (Annexe A §5) — à déclarer dans `src/index.css (bloc @theme)` :

Token	Rôle	Couleur proposée
`navy`	Confiance, titres, en-tête	#0B1F3A
`brand`	Boutons et liens d'action	#1D4ED8
`info`	Badges et états informatifs	#14B8A6
`warn`	Limites et avertissements uniquement	#D97706

Les codes couleur sont à ajuster sur la planche de référence du prototype.

4. Étapes de développement à partir des prototypes

Le frontend démarre dès l'heure 4 sur des données fictives (mocks) au format de l'API (§26), puis bascule sur l'API réelle à partir de l'heure 30 : il n'attend jamais le backend.



Roadmap frontend · 4 phases sur 48 h

Le tableau ci-dessous détaille chaque étape, ses heures indicatives et les fonctions du cahier des charges qu'elle couvre.

Étape	Heures	Livrable	Fonctions
0. Initialisation	0–4	Projet Vite + React + Tailwind + Router, mocks	—
1. Layout	4–8	Header, navigation, footer, `LegalDisclaimer`	F10
2. Écran 1 — Accueil	8–10	Promesse, « Analyser un cas », accès recherche	—
3. Écran 2 — Assistant IA	10–14	Zone de description, exemple, rappel données perso	F01, F02
4. Écran 3 — Résultat	14–20	Cartes Faits / Qualifications / Scénarios / Sources / Limites	F03, F04, F07, F08
5. Écran 4 — Détail article	20–23	Texte officiel + « Expliquer simplement »	F06, F07
6. Base juridique	23–26	Recherche, filtres, liste d'articles	F05

Étape	Heures	Livrable	Fonctions
7. États	26–30	Chargement, erreur, clarification, refus contrôlé	F08, F09
8. API réelle	30–38	`VITE_USE_MOCKS=false`, tests de bout en bout	Toutes
9. Démo	38–48	Scénario §39, responsive, déploiement	—

Points de contrôle par écran (Annexe A §3)

1. **Accueil** — une seule action principale (« Analyser un cas »), lien secondaire « Rechercher dans le droit haïtien », corpus couvert affiché.
2. **Assistant IA** — zone de texte avec exemple, bouton « Utiliser un exemple », rappel « Ne saisissez ni nom, ni téléphone, ni adresse » (§30).
3. **Résultat** — trois zones distinctes : *Faits fournis*, *Analyse IA* (badge « Généré par l'IA »), *Texte officiel*. Champ « Ajouter un fait » pour la démo (§39, étape 7).
4. **Détail d'article** — texte original toujours au-dessus de l'explication simplifiée.
5. **Partout** — navigation clavier, libellés explicites, mise en page mobile d'abord.

Correspondance réponse API → composants React

Champ JSON (§26)	Composant
`case_summary`	`<FactsCard />`
`possible_qualifications[]`	`<QualificationCard />`
`possible_qualifications[].sources[]`	`<Link to="/base-juridique/:id">`
`scenarios[]`	`<ScenarioTabs />`
`limitations[]`	`<LimitsBox />`

5. Labs au format CodePath (React JS)

Chaque lab suit le modèle CodePath : objectif, fonctionnalités requises, bonus, milestones avec code, vérification et rendu (GIF + commit). Tout le code est en **JavaScript + JSX**, régénérable avec les prompts de la section 6.

README à rendre pour chaque lab

```
# Lab N — <Titre>
Submitted by: <nom>
Time spent: <heures>

## Required Features
- [x] ...
## Stretch Features
- [ ] ...
## Video Walkthrough
<GIF du fonctionnement>
## Notes
Difficultés rencontrées, décisions prises.
```

Lab 0 — Initialiser le projet React

Objectif : un projet Vite + React qui démarre avec Tailwind, React Router et la palette JuriPredict.

Fonctionnalités requises

- [ ] Projet créé avec le template `react` (JavaScript)
- [ ] Tailwind CSS installé et palette `navy / brand / info / warn` déclarée
- [ ] React Router et lucide-react installés
- [ ] Fichier `.env` avec `VITE_API_URL` et `VITE_USE_MOCKS`

Bonus

- [ ] Prettier configuré
- [ ] Déploiement de preview (Vercel ou Netlify)

Milestone 1 — Création du projet

```
npm create vite@latest juripredict-frontend -- --template react
cd juripredict-frontend
npm install
npm install react-router-dom lucide-react
npm install tailwindcss @tailwindcss/vite
```

Milestone 2 — `vite.config.js`

```
import { defineConfig } from "vite";
import react from "@vitejs/plugin-react";
import tailwindcss from "@tailwindcss/vite";

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

Milestone 3 — `src/index.css` (Tailwind + palette)

```
@import "tailwindcss";

@theme {
  --color-navy: #0b1f3a;
  --color-brand: #1d4ed8;
  --color-info: #14b8a6;
  --color-warn: #d97706;
}
```

Milestone 4 — `.env`

```
VITE_API_URL=http://localhost:8000
VITE_USE_MOCKS=true
```

Vérification : `npm run dev` ouvre http://localhost:5173 ; une classe `bg-navy` colore bien un élément.

Lab 1 — Client API et données fictives

Objectif : coder une seule fois les appels à l'API (§25–26), avec une bascule mocks / API réelle.

Fonctionnalités requises

- [ ] `analyzeCase`, `searchArticles`, `getArticle`, `explainText` dans `services/api.js`
- [ ] Mocks réalistes reprenant le cas de démonstration (§39)
- [ ] Erreur claire si l'API répond avec un code d'erreur

Bonus

- [ ] Délai d'attente (timeout) de 30 secondes avec `AbortController`
- [ ] Mock « informations insuffisantes » pour tester F09

Milestone 1 — `src/services/mocks.js` (numéros d'articles laissés en `XXX` comme dans le cahier des charges : ne jamais inventer de numéro)

```
export const mockAnalysis = {
  case_summary:
    "Une personne aurait pris un bien appartenant à une autre personne sans
son autorisation.",
  possible_qualifications: [
    {
      name: "Vol",
      relevance: "high",
      sources: [{ document: "Code pénal", article: "XXX", article_id: "cp-1"
}],
    },
    {
      name: "Vol accompagné de circonstances particulières",
      relevance: "medium",
      sources: [{ document: "Code pénal", article: "XXX", article_id: "cp-2"
}],
    },
  ],
  scenarios: [
    { title: "Scénario A", description: "Les éléments fournis correspondent à
certaines conditions du texte X." },
    { title: "Scénario B", description: "Si une violence est établie, une
autre disposition pourrait devenir pertinente." },
    { title: "Scénario C", description: "Si l'absence d'autorisation n'est
pas établie, la qualification pourrait différer." },
  ],
  limitations: ["Informations insuffisantes sur l'usage éventuel de la
violence."],
  clarification_questions: ["Une violence a-t-elle été utilisée ?"],
};

export const mockArticles = [
  {
    id: "cp-1",
    document_title: "Code pénal haïtien",
    article_number: "XXX",
    title: "Du vol",
    chapter: "Chapitre à compléter",
    content: "Texte officiel de l'article, copié depuis le corpus vérifié.",
    file_url: "#",
    relevance: "high",
  },
];
```

Milestone 2 — `src/services/api.js`

```
import { mockAnalysis, mockArticles } from "./mocks";

const API_URL = import.meta.env.VITE_API_URL;
const USE_MOCKS = import.meta.env.VITE_USE_MOCKS === "true";
const wait = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

async function request(path, options = {}) {
  const res = await fetch(`${API_URL}${path}`, {
    headers: { "Content-Type": "application/json" },
    ...options,
  });
  if (!res.ok) throw new Error(`Erreur API ${res.status}`);
  return res.json();
}

// POST /api/v1/analyze — nom du champ "query" à confirmer avec le backend
export async function analyzeCase(facts) {
  if (USE_MOCKS) {
    await wait(800);
    return mockAnalysis;
  }
  return request("/api/v1/analyze", {
    method: "POST",
    body: JSON.stringify({ query: facts }),
  });
}

// POST /api/v1/search — recherche sémantique
export async function searchArticles(query) {
  if (USE_MOCKS) {
    await wait(400);
    const q = query.toLowerCase();
    return mockArticles.filter((a) =>
      `${a.title} ${a.content}`.toLowerCase().includes(q)
    );
  }
  return request("/api/v1/search", {
    method: "POST",
    body: JSON.stringify({ query }),
  });
}

// GET /api/v1/articles/{id}
export async function getArticle(id) {
  if (USE_MOCKS) {
    await wait(300);
```

```
    return mockArticles.find((a) => a.id === id) ?? null;
  }
  return request(`/api/v1/articles/${id}`);
}

// POST /api/v1/explain — simplification d'un texte juridique
export async function explainText(text) {
  if (USE_MOCKS) {
    await wait(600);
    return { explanation: "Explication simplifiée générée par l'IA
(exemple)." };
  }
  return request("/api/v1/explain", {
    method: "POST",
    body: JSON.stringify({ text }),
  });
}
```

Vérification : dans la console du navigateur, `analyzeCase("test")` renvoie le mock après 0,8 s.

Lab 2 — Routes, layout et avertissement juridique

Objectif : la coquille commune à toutes les pages, avec l'avertissement F10 visible sur chaque analyse.

Fonctionnalités requises

- [ ] Routes : `/`, `/assistant`, `/resultat`, `/base-juridique`, `/base-juridique/:id`, `/ressources`
- [ ] Header avec navigation et lien actif surligné
- [ ] `AnalysisContext` pour partager l'analyse entre pages
- [ ] Composant `LegalDisclaimer` (texte F10 mot pour mot)

Bonus

- [ ] Menu burger sur mobile
- [ ] Lien « Aller au contenu » pour la navigation clavier

Milestone 1 — `src/main.jsx`

```
import React from "react";
import ReactDOM from "react-dom/client";
import { BrowserRouter } from "react-router-dom";
import App from "./App";
import { AnalysisProvider } from "./context/AnalysisContext";
import "./index.css";

ReactDOM.createRoot(document.getElementById("root")).render(
  <React.StrictMode>
    <BrowserRouter>
      <AnalysisProvider>
        <App />
      </AnalysisProvider>
    </BrowserRouter>
  </React.StrictMode>
);
```

Milestone 2 — `src/App.jsx`

```
import { Routes, Route } from "react-router-dom";
import Layout from "./components/layout/Layout";
import HomePage from "./pages/HomePage";
import AssistantPage from "./pages/AssistantPage";
import ResultPage from "./pages/ResultPage";
import SearchPage from "./pages/SearchPage";
import ArticlePage from "./pages/ArticlePage";
import ResourcesPage from "./pages/ResourcesPage";

export default function App() {
  return (
    <Routes>
      <Route element={<Layout />}>
        <Route path="/" element={<HomePage />} />
        <Route path="/assistant" element={<AssistantPage />} />
        <Route path="/resultat" element={<ResultPage />} />
        <Route path="/base-juridique" element={<SearchPage />} />
        <Route path="/base-juridique/:id" element={<ArticlePage />} />
        <Route path="/ressources" element={<ResourcesPage />} />
      </Route>
    </Routes>
  );
}
```

Milestone 3 — `src/context/AnalysisContext.jsx`

```
import { createContext, useContext, useState } from "react";

const AnalysisContext = createContext(null);

export function AnalysisProvider({ children }) {
  const [facts, setFacts] = useState("");
  const [result, setResult] = useState(null);
  return (
    <AnalysisContext.Provider value={{ facts, setFacts, result, setResult }}>
      {children}
    </AnalysisContext.Provider>
  );
}

export const useAnalysis = () => useContext(AnalysisContext);
```

Milestone 4 — `src/components/layout/Layout.jsx`

```
import { NavLink, Outlet } from "react-router-dom";
import { Scale } from "lucide-react";

const links = [
  { to: "/", label: "Accueil" },
  { to: "/assistant", label: "Assistant IA" },
  { to: "/base-juridique", label: "Base juridique" },
  { to: "/ressources", label: "Ressources" },
];

export default function Layout() {
  return (
    <div className="min-h-screen bg-slate-50 text-slate-800">
      <header className="bg-navy text-white">
        <nav className="mx-auto flex max-w-5xl items-center gap-6 p-4" aria-
label="Navigation principale">
          <span className="flex items-center gap-2 font-semibold">
            <Scale size={20} aria-hidden="true" /> JuriPredict AI
          </span>
          {links.map((l) => (
            <NavLink key={l.to} to={l.to} end
              className={({ isActive }) => (isActive ? "text-info underline"
: "hover:text-info")}>
              {l.label}
            </NavLink>
          ))}
        </nav>
      </header>
```

```
      <main id="contenu" className="mx-auto max-w-5xl p-4">
        <Outlet />
      </main>
    </div>
  );
}
```

Milestone 5 — `src/components/layout/LegalDisclaimer.jsx`

```
import { AlertTriangle } from "lucide-react";

export default function LegalDisclaimer() {
  return (
    <aside role="note" className="flex gap-3 rounded-lg border border-warn/40
bg-warn/5 p-4 text-sm">
      <AlertTriangle className="mt-0.5 shrink-0 text-warn" size={18} aria-
hidden="true" />
      <p>
        JuriPredict est un outil d'information et d'analyse juridique assisté
par
        intelligence artificielle. Les résultats peuvent contenir des erreurs
ou être
        incomplets. Ils ne constituent pas un avis juridique professionnel et
ne
        remplacent pas les conseils d'un avocat ou la décision d'une
juridiction
        compétente.
      </p>
    </aside>
  );
}
```

Vérification : chaque lien du menu change de page sans recharger le navigateur.

Lab 3 — Écran 2 : Assistant IA

Objectif : l'utilisateur décrit sa situation et lance l'analyse.

Fonctionnalités requises

- [ ] Zone de texte avec exemple en placeholder
- [ ] Bouton « Utiliser un exemple »
- [ ] Rappel : ne pas saisir de données personnelles
- [ ] Bouton « Analyser » désactivé sous 20 caractères
- [ ] État de chargement, puis redirection vers `/resultat`

Bonus

- [ ] Compteur de caractères
- [ ] Alerte si un numéro de téléphone est détecté dans le texte

Milestone 1 — `src/pages/AssistantPage.jsx`

```
import { useState } from "react";
import { useNavigate } from "react-router-dom";
import { analyzeCase } from "../services/api";
import { useAnalysis } from "../context/AnalysisContext";

const EXEMPLE =
  "Une personne est entrée dans une maison sans autorisation pendant la nuit
et a emporté plusieurs objets.";

export default function AssistantPage() {
  const { facts, setFacts, setResult } = useAnalysis();
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState(null);
  const navigate = useNavigate();

  async function handleSubmit(e) {
    e.preventDefault();
    setLoading(true);
    setError(null);
    try {
      const data = await analyzeCase(facts);
      setResult(data);
      navigate("/resultat");
    } catch {
      setError("L'analyse a échoué. Réessayez dans un instant.");
    } finally {
      setLoading(false);
    }
  }

  return (
    <form onSubmit={handleSubmit} className="mx-auto max-w-2xl space-y-4">
      <h1 className="text-2xl font-semibold text-navy">Décrivez votre
situation juridique</h1>
      <p className="rounded bg-warn/10 p-2 text-sm">
        Ne saisissez ni nom complet, ni téléphone, ni adresse.
      </p>
      <label htmlFor="facts" className="sr-only">Description des
faits</label>
      <textarea id="facts" rows={8} value={facts} placeholder={EXEMPLE}
```

```
        onChange={(e) => setFacts(e.target.value)}
        className="w-full rounded-lg border p-3 focus:outline-2
focus:outline-brand" />
      <div className="flex gap-2">
        <button type="button" onClick={() => setFacts(EXEMPLE)}
          className="rounded-lg border px-4 py-2">Utiliser un
exemple</button>
        <button type="submit" disabled={facts.trim().length < 20 || loading}
          className="rounded-lg bg-brand px-4 py-2 text-white
disabled:opacity-50">
          {loading ? "Analyse en cours…" : "Analyser"}
        </button>
      </div>
      {error && <p role="alert" className="text-sm text-red-600">{error}</p>}
    </form>
  );
}
```

Vérification : avec `VITE_USE_MOCKS=true`, « Analyser » mène à `/resultat` après environ 1 seconde.

Lab 4 — Écran 1 : Accueil

Objectif : une page d'accueil avec une promesse claire et une seule action principale.

Fonctionnalités requises

- [ ] Titre « Le droit haïtien, à portée de main » + phrase de présentation (§47)
- [ ] Bouton principal « Analyser un cas » vers `/assistant`
- [ ] Lien secondaire « Rechercher dans le droit haïtien »
- [ ] Trois cartes : Sources vérifiables, Analyse par scénarios, Explication simple

Milestone 1 — `src/pages/HomePage.jsx`

```
import { Link } from "react-router-dom";
import { BookOpen, GitBranch, MessageSquareText } from "lucide-react";

const features = [
  { icon: BookOpen, title: "Sources vérifiables", text: "Chaque réponse
renvoie vers l'article original." },
  { icon: GitBranch, title: "Analyse par scénarios", text: "Plusieurs
qualifications possibles, jamais une décision." },
  { icon: MessageSquareText, title: "Explication simple", text: "Le texte
juridique reformulé en langage clair." },
];
```

```
export default function HomePage() {
  return (
    <section className="space-y-10 py-8">
      <div className="rounded-2xl bg-navy p-8 text-white">
        <h1 className="text-3xl font-bold">Le droit haïtien, à portée de
main.</h1>
        <p className="mt-3 max-w-xl text-slate-200">
          Comprendre le droit haïtien, explorer les scénarios, vérifier les
sources.
        </p>
        <div className="mt-6 flex flex-wrap gap-3">
          <Link to="/assistant" className="rounded-lg bg-brand px-5 py-3
font-medium">Analyser un cas</Link>
          <Link to="/base-juridique" className="rounded-lg border border-
white/40 px-5 py-3">
            Rechercher dans le droit haïtien
          </Link>
        </div>
      </div>
      <div className="grid gap-4 md:grid-cols-3">
        {features.map(({ icon: Icon, title, text }) => (
          <article key={title} className="rounded-xl border bg-white p-5">
            <Icon className="text-info" aria-hidden="true" />
            <h2 className="mt-3 font-semibold text-navy">{title}</h2>
            <p className="mt-1 text-sm text-slate-600">{text}</p>
          </article>
        ))}
      </div>
    </section>
  );
}
```

Vérification : sur mobile (375 px), les boutons passent l'un sous l'autre sans déborder.

Lab 5 — Écran 3 : Résultat d'analyse

Objectif : afficher l'analyse en cartes en séparant faits fournis, analyse IA et sources officielles.

Fonctionnalités requises

- [ ] Carte « Faits fournis » (texte saisi)
- [ ] Cartes « Qualifications possibles » avec niveau de pertinence (F03, F08)
- [ ] Onglets « Scénarios » A / B / C (F04)
- [ ] Chaque source cliquable vers `/base-juridique/:id` (F07)
- [ ] Encadré « Limites » + `LegalDisclaimer`

- [ ] Badge « Généré par l'IA » sur les zones générées

Bonus

- [ ] Champ « Ajouter un fait » qui relance l'analyse (démo §39, étape 7)
- [ ] Animation légère sur les cartes qui changent

Milestone 1 — `src/components/analysis/QualificationCard.jsx`

```
import { Link } from "react-router-dom";

const LABELS = { high: "Pertinence élevée", medium: "Pertinence moyenne",
low: "Pertinence faible" };

export default function QualificationCard({ qualification, index }) {
  const { name, relevance, sources } = qualification;
  return (
    <article className="rounded-xl border bg-white p-4">
      <header className="flex items-center justify-between gap-2">
        <h3 className="font-semibold text-navy">
          Qualification potentielle #{index + 1} — {name}
        </h3>
        <span className="rounded-full bg-info/15 px-3 py-1 text-xs text-
navy">{LABELS[relevance]}</span>
      </header>
      <div className="mt-3 flex flex-wrap gap-2">
        {sources.map((s) => (
          <Link key={`${s.document}-${s.article}`} to={`/base-
juridique/${s.article_id}`}
            className="rounded border px-2 py-1 text-sm text-brand
hover:underline">
            {s.document}, article {s.article}
          </Link>
        ))}
      </div>
    </article>
  );
}
```

Milestone 2 — `src/components/analysis/ScenarioTabs.jsx`

```
import { useState } from "react";

export default function ScenarioTabs({ scenarios }) {
  const [active, setActive] = useState(0);
  if (!scenarios.length) return null;
  return (
    <div>
      <div role="tablist" className="flex gap-2 border-b">
        {scenarios.map((s, i) => (
          <button key={s.title} role="tab" aria-selected={active === i}
onClick={() => setActive(i)}
            className={`px-3 py-2 text-sm ${active === i ? "border-b-2
border-brand font-semibold" : ""}`}>
            {s.title}
          </button>
        ))}
      </div>
      <p role="tabpanel" className="p-3 text-sm">
{scenarios[active].description}</p>
    </div>
  );
}
```

Milestone 3 — `src/pages/ResultPage.jsx`

```
import { useState } from "react";
import { Link } from "react-router-dom";
import { useAnalysis } from "../context/AnalysisContext";
import { analyzeCase } from "../services/api";
import QualificationCard from "../components/analysis/QualificationCard";
import ScenarioTabs from "../components/analysis/ScenarioTabs";
import LegalDisclaimer from "../components/layout/LegalDisclaimer";

const AiBadge = () => (
  <span className="rounded-full border px-2 py-0.5 text-xs text-slate-
500">Généré par l'IA</span>
);

export default function ResultPage() {
  const { facts, setFacts, result, setResult } = useAnalysis();
  const [newFact, setNewFact] = useState("");
  const [loading, setLoading] = useState(false);

  if (!result) {
    return <p>Aucune analyse. <Link to="/assistant" className="text-
brand">Commencer</Link></p>;
```

```
  }

  async function addFact() {
    const updated = `${facts} ${newFact}`.trim();
    setLoading(true);
    setFacts(updated);
    setResult(await analyzeCase(updated));
    setNewFact("");
    setLoading(false);
  }

  return (
    <div className="mx-auto max-w-3xl space-y-6">
      <section>
        <h2 className="font-semibold text-navy">Faits fournis</h2>
        <p className="mt-2 rounded-lg bg-white p-3 text-sm">{facts}</p>
        <div className="mt-2 flex gap-2">
          <input value={newFact} onChange={(e) => setNewFact(e.target.value)}
            placeholder="Ajouter un fait (ex. : la personne aurait utilisé la
violence)"
            className="flex-1 rounded-lg border p-2 text-sm" aria-
label="Ajouter un fait" />
          <button onClick={addFact} disabled={!newFact.trim() || loading}
            className="rounded-lg bg-brand px-3 text-sm text-white
disabled:opacity-50">
            {loading ? "Réanalyse…" : "Réanalyser"}
          </button>
        </div>
      </section>

      <section className="space-y-3">
        <div className="flex items-center gap-2">
          <h2 className="font-semibold text-navy">Qualifications
possibles</h2><AiBadge />
        </div>
        <p className="text-sm">{result.case_summary}</p>
        {result.possible_qualifications.map((q, i) => (
          <QualificationCard key={q.name} qualification={q} index={i} />
        ))}
      </section>

      <section>
        <div className="flex items-center gap-2">
          <h2 className="font-semibold text-navy">Scénarios</h2><AiBadge />
        </div>
        <ScenarioTabs scenarios={result.scenarios} />
```

```
      </section>

      {result.limitations.length > 0 && (
        <section className="rounded-lg border border-warn/40 bg-warn/5 p-4
text-sm">
          <h2 className="font-semibold">Limites de l'analyse</h2>
          <ul className="mt-1 list-disc pl-5">
            {result.limitations.map((l) => <li key={l}>{l}</li>)}
          </ul>
        </section>
      )}

      <LegalDisclaimer />
    </div>
  );
}
```

Vérification : les trois zones (faits, IA, sources) se distinguent d'un coup d'œil ; chaque source ouvre une page d'article.

Lab 6 — Écran 4 : Détail d'un article

Objectif : montrer le texte officiel puis, à la demande, une explication simplifiée (F06).

Fonctionnalités requises

- [ ] Document, numéro d'article, chapitre
- [ ] Texte original dans un encadré « Texte officiel »
- [ ] Bouton « Expliquer simplement » ; explication affichée **sous** le texte original
- [ ] Lien « Télécharger le document source »

Bonus

- [ ] Bouton « Copier la référence »
- [ ] Articles précédent / suivant

Milestone 1 — `src/pages/ArticlePage.jsx`

```
import { useEffect, useState } from "react";
import { useParams } from "react-router-dom";
import { getArticle, explainText } from "../services/api";

export default function ArticlePage() {
  const { id } = useParams();
  const [article, setArticle] = useState(null);
  const [explanation, setExplanation] = useState("");
  const [explaining, setExplaining] = useState(false);
```

```
  useEffect(() => {
    getArticle(id).then(setArticle);
  }, [id]);

  async function handleExplain() {
    setExplaining(true);
    const { explanation } = await explainText(article.content);
    setExplanation(explanation);
    setExplaining(false);
  }

  if (!article) return <p>Chargement de l'article…</p>;

  return (
    <article className="mx-auto max-w-3xl space-y-4">
      <p className="text-sm text-slate-500">{article.document_title} ·
{article.chapter}</p>
      <h1 className="text-2xl font-semibold text-navy">Article
{article.article_number} — {article.title}</h1>
      <blockquote className="rounded-lg border-l-4 border-navy bg-white p-4
font-serif">
        <span className="mb-2 block text-xs uppercase tracking-wide text-
slate-500">Texte officiel</span>
        {article.content}
      </blockquote>
      <div className="flex gap-3">
        <button onClick={handleExplain} disabled={explaining}
          className="rounded-lg bg-brand px-4 py-2 text-white
disabled:opacity-50">
          {explaining ? "Explication…" : "Expliquer simplement"}
        </button>
        <a href={article.file_url} className="rounded-lg border px-4 py-
2">Télécharger le document source</a>
      </div>
      {explanation && (
        <section className="rounded-lg bg-info/10 p-4 text-sm">
          <span className="text-xs text-slate-500">Explication générée par
l'IA</span>
          <p className="mt-1">{explanation}</p>
        </section>
      )}
    </article>
  );
}
```

Vérification : après le clic, le texte officiel reste affiché tel quel au-dessus de l'explication.

Lab 7 — Base juridique (recherche)

Objectif : chercher un mot-clé (vol, propriété, contrat…) et ouvrir les articles (F05).

Fonctionnalités requises

- [ ] Barre de recherche, validation avec Entrée
- [ ] Filtre par document (Constitution, Code pénal, Code civil)
- [ ] Ligne de résultat : article, titre, extrait, document, pertinence
- [ ] État vide : « Aucun article trouvé dans le corpus disponible »
- [ ] Requête gardée dans l'URL avec `useSearchParams` (`?q=vol`)

Bonus

- [ ] Mot-clé surligné dans l'extrait
- [ ] Pagination

Vérification : rechercher « vol » affiche une liste ; chaque ligne ouvre l'Écran 4. Le code est à générer avec le prompt Lab 7 de la section 6.

Lab 8 — États, clarification et refus contrôlé

Objectif : gérer tout ce qui n'est pas le « cas heureux ».

Fonctionnalités requises

- [ ] Squelette de chargement (cartes grises animées avec `animate-pulse`)
- [ ] Si `clarification_questions` existe : questions affichées + champ de réponse (F09)
- [ ] Si aucune qualification : « Les documents disponibles ne permettent pas d'établir cette information avec suffisamment de certitude. »
- [ ] Erreur réseau : message clair + bouton « Réessayer »

Vérification : tester les 6 cas de la Phase 6 (questions simples, cas ambigus, hors corpus, hallucinations, sources incorrectes, informations insuffisantes) avec un mock dédié pour chacun.

6. Prompts IA en programmation en binôme

L'IA joue le rôle du collègue de binôme, comme dans les sessions CodePath : un **Driver** écrit le code, un **Navigator** relit, questionne et guide. On échange les rôles à chaque milestone.

Mode	Toi	L'IA	Quand l'utiliser
A — IA Driver	Navigator : tu fixes le cap et tu relis	Écrit le code, explique chaque choix	Nouveau composant, code répétitif
B — IA Navigator	Driver : tu écris le code	Relit, pose des questions, donne des indices sans tout réécrire	Apprendre, consolider, corriger
C — Débogage à deux	Tu décris le bug	Formule des hypothèses, propose des tests	Erreur console, écran blanc

Règles de la session

1. Toujours commencer par le prompt de contexte ci-dessous.
2. Un milestone = un prompt. Ne jamais demander tout le projet d'un coup.
3. Lire et exécuter le code avant de passer au milestone suivant.
4. Changer de mode (A ↔ B) à chaque milestone.
5. Ne jamais coller de données personnelles réelles ni de clés API dans un prompt.

Prompt de contexte (à coller en premier)

```
Tu es mon binôme de programmation pour le projet JuriPredict AI, un assistant
juridique IA pour le droit haïtien.
Je suis le développeur frontend.

Stack : React 18 en JavaScript (JSX, pas de TypeScript), Vite, React Router,
Tailwind CSS v4, lucide-react, fetch natif.
Palette Tailwind : navy (confiance), brand (actions), info (badges), warn
(limites et avertissements uniquement).
Structure : src/pages, src/components/{layout,analysis,search},
src/context/AnalysisContext.jsx, src/services/{api.js,mocks.js}.
API FastAPI : POST /api/v1/analyze, POST /api/v1/search, GET
/api/v1/articles/{id}, POST /api/v1/explain.
Réponse d'analyse : { case_summary, possible_qualifications[{name, relevance,
sources[{document, article, article_id}]}], scenarios[{title, description}],
limitations[] }.

Règles produit non négociables :
- séparer visuellement faits fournis, analyse IA et texte juridique officiel
;
- toute affirmation juridique est reliée à une source cliquable ;
- afficher l'avertissement juridique sur chaque analyse ;
- ne jamais présenter un résultat comme une décision certaine ;
- accessibilité : navigation clavier, libellés explicites, mobile d'abord.

On travaille en programmation en binôme, un milestone à la fois. Attends mes
instructions.
```

Prompts mode A — l'IA écrit (Driver)

Lab 0–1 — Setup et client API

```
Mode A : tu es Driver, je suis Navigator.
Milestone : Lab 1 — créer src/services/mocks.js et src/services/api.js.
- Une variable VITE_USE_MOCKS bascule entre mocks et API réelle.
- Fonctions : analyzeCase, searchArticles, getArticle, explainText.
- Numéros d'articles en "XXX" dans les mocks : n'invente aucun numéro réel.
Avant d'écrire le code, annonce ton plan en 3 points et attends mon "OK".
Après le code, explique chaque fonction en une phrase et dis-moi comment la
tester.
```

Lab 5 — Écran Résultat

```
Mode A : tu es Driver.
Milestone : Lab 5 — src/pages/ResultPage.jsx avec QualificationCard et
ScenarioTabs.
Contraintes : trois zones visuellement distinctes (Faits fournis / Analyse IA
avec badge "Généré par l'IA" / Sources),
encadré Limites en couleur warn, LegalDisclaimer en bas, onglets accessibles
(role=tablist, aria-selected).
Données : useAnalysis() depuis AnalysisContext.
Fais-moi d'abord un schéma ASCII de la page, puis le code, puis 3 questions
que je dois me poser en relisant.
```

Lab 7 — Base juridique

```
Mode A : tu es Driver.
Milestone : Lab 7 — src/pages/SearchPage.jsx.
- Barre de recherche (Entrée valide), requête gardée dans l'URL avec
useSearchParams.
- Filtre par document : Constitution, Code pénal, Code civil.
- Ligne de résultat : article, titre, extrait, document, pertinence ; lien
vers /base-juridique/:id.
- États : chargement, vide ("Aucun article trouvé dans le corpus
disponible"), erreur.
Utilise searchArticles() de services/api.js. Découpe en composants SearchBar
et ArticleItem.
```

Prompts mode B — j'écris, l'IA relit (Navigator)

Relecture d'un composant

```
Mode B : je suis Driver, tu es Navigator.
Voici mon composant [nom du fichier] :
[coller le code]

Ne réécris pas le code. Fais comme un binôme CodePath :
1. Dis-moi ce qui fonctionne bien.
2. Pose-moi au maximum 3 questions qui m'aident à trouver les problèmes moi-
même.
3. Vérifie les règles produit (séparation IA / texte officiel, sources,
avertissement, accessibilité).
4. Donne un seul indice si je bloque, pas la solution complète.
```

Lab 3 — Assistant IA (je code, tu guides)

```
Mode B : je vais coder AssistantPage.jsx moi-même.
Guide-moi étape par étape : donne-moi seulement la prochaine petite étape
(état, puis formulaire, puis appel API, puis redirection).
Attends que je colle mon code avant de passer à l'étape suivante.
```

Prompts mode C — débogage à deux

```
Mode C : débogage en binôme.
Ce que j'attends : [comportement attendu]
Ce qui se passe : [comportement réel]
Message d'erreur : [copier la console]
Fichier concerné : [coller le code]

Propose 3 hypothèses classées de la plus probable à la moins probable,
et pour chacune un test rapide (console.log, onglet Réseau, React DevTools)
avant toute correction.
```

Prompt de régénération complète d'un lab

```
Régénère le Lab [N] au format CodePath pour JuriPredict AI en React JS :
Objectif, Required Features (cases à cocher), Stretch Features, Milestones
avec le code complet de chaque fichier,
étape de Vérification, et le README à rendre (Submitted by, Time spent, Video
Walkthrough, Notes).
Respecte le prompt de contexte. Code en JavaScript + JSX uniquement.
```

7. Intégrations futures côté frontend

Après le hackathon, chaque version de la feuille de route (§42) ajoute des écrans ou des composants React ; l'architecture `services/` + `context/` les accueille sans réécrire le MVP.

Version	Fonction prévue (§42)	Travail frontend React	Outils suggérés
V2	Comptes utilisateurs	Pages Connexion / Inscription, routes protégées, jeton JWT dans les appels API	React Router (routes protégées), Context Auth
V2	Historique des analyses	Page « Mes analyses », reprise d'une analyse	TanStack Query (cache)

Version	Fonction prévue (§42)	Travail frontend React	Outils suggérés
V2	Recherche avancée + jurisprudence	Filtres multiples, type de source « Jurisprudence »	Composants de filtres réutilisables
V2	Code de procédure pénale	Nouveau filtre de document, aucune refonte	—
V3	Support français / créole haïtien	Interface bilingue, sélecteur de langue	react-i18next
V3	Application mobile	Version mobile qui réutilise `services/api.js`	PWA, puis React Native
V3	Analyse de documents	Zone de dépôt de fichier PDF, suivi de l'envoi	Input fichier + FormData
V3	Comparaison de textes	Vue côte à côte de deux articles	Grille CSS 2 colonnes
V3	Outils pour étudiants en droit	Fiches, quiz, favoris	Composants dédiés
V4	Graphes de relations entre articles	Visualisation interactive des liens	React Flow ou D3
V4	Analyse de dossiers complexes	Tableau de bord multi-faits, chronologie	Dashboard
V4	API publique	Portail développeur (documentation, clés)	—

Intégrations techniques à prévoir dès maintenant

- **Réponses en continu (streaming)** : afficher l'analyse au fur et à mesure si le backend l'envoie par morceaux ; prévoir un composant qui accepte un texte partiel.
- **Gestion des clés et de la sécurité (§29)** : aucune clé d'API IA dans le frontend ; seules les variables `VITE_` publiques y figurent.
- **Anonymisation (§30)** : détection côté client des numéros de téléphone et adresses avant envoi, avec alerte à l'utilisateur.
- **Mesure des KPIs (§37)** : mesurer le temps de réponse affiché et proposer un « Cette explication est-elle claire ? oui / non » (KPI 5).

8. Checklist qualité et livraison

Le frontend est prêt pour la démo quand le scénario §39 passe de bout en bout sur l'API réelle, sur ordinateur et sur téléphone.

Produit

- [ ] Les 4 écrans prototypes sont fidèles à la planche de référence
- [ ] Faits / analyse IA / texte officiel sont séparés visuellement
- [ ] Chaque qualification a au moins une source cliquable
- [ ] L'avertissement F10 est visible sur Résultat et Détail d'article
- [ ] Le mot « prédiction » n'apparaît nulle part comme certitude

Technique

- [ ] `VITE_USE_MOCKS=false` : tous les appels passent par l'API FastAPI
- [ ] Aucune erreur dans la console du navigateur
- [ ] Chargement, erreur, clarification et refus contrôlé testés
- [ ] `npm run build` réussit ; déploiement en ligne vérifié

Accessibilité et responsive

- [ ] Tout le parcours se fait au clavier (Tab, Entrée)
- [ ] Contraste lisible, libellés sur tous les champs
- [ ] Affichage correct à 375 px, 768 px et 1280 px

Scénario de démo (§39)

1. Ouvrir JuriPredict (Accueil).
2. Cliquer « Analyser un cas » et saisir : « Une personne est accusée d'avoir pris un bien appartenant à une autre personne sans son autorisation. »
3. Montrer les qualifications et les scénarios.
4. Cliquer « Voir la source » : l'article original s'affiche.
5. Ajouter le fait « La personne aurait utilisé la violence » et réanalyser.
6. Montrer que les dispositions pertinentes évoluent avec les faits.

Convention Git de l'équipe

- Une branche par lab : `feat/lab-5-resultat`
- Un commit par milestone : `feat(resultat): ajoute ScenarioTabs`
- Pull request relue par un autre membre avant fusion dans `main`